LEVEL 1 · STUDY NOTES

🧠 LLM 101 — Concepts + Interview Prep

Level 1 · Modules 1.1 – 1.12 · CampusX AI Engineering Roadmap 2026 · Roman Urdu

> Maqsad: LLM kaise kaam karta hai, kyun aisa behave karta hai, aur kaam ke liye sahi model kaise chunein — is ka **mazboot mental model**. Interview weight **8/10** · Job weight **7/10**

FEHRIST

1.1 What an LLM Actually Is · 1.2 Tokenization · 1.3 Embeddings, Attention & Transformers · 1.4 How LLMs Learn · 1.5 Inference · 1.6 Sampling · 1.7 Reasoning Models · 1.8 Context Window · 1.9 Capabilities & Failure Modes · 1.10 Model Landscape · 1.11 Benchmarks · 1.12 Choosing the Right Model · **Revision Sheet + Rapid-Fire**

1.1 What an LLM Actually Is

📘 Concept

LLM (Large Language Model) ka asal kaam sirf ek hai: **ab tak ke text ko dekh kar agla token predict karna.** Bas. Baqi sab — chat, coding, summary — isi ek kaam ko bar bar dohrane se nikalta hai.

Socho phone ke keyboard ka autocomplete — "mein ghar..." likho to woh "ja raha hoon" suggest karta hai. LLM wahi cheez hai, magar arbon alfaaz par trained aur bohot gehri samajh ke saath.

Generation Loop

```
Predict  →  Choose  →  Append  →  Repeat   (jab tak stop token ya max limit na aaye)
```

1. **Predict:** har mumkin agle token ki probability nikalo
2. **Choose:** un mein se ek token chuno (sampling — 1.6 mein)
3. **Append:** us token ko text ke aakhir mein jor do
4. **Repeat:** naye, lambe text par phir se

LLM kya NAHI hai

Ghalat soch	Haqeeqat
Database	Facts store nahi karta; patterns weights mein phaile hue hain
Search engine	Internet nahi dekhta (jab tak tool na diya jaye)
Deterministic program	Ek hi input par har bar alag jawab mumkin
Sach batane wala	**'Munasib' text** banata hai, **'sach'** verify nahi karta

Ek model, bohot se kaam

Summary, translation, coding, Q&A, classification — sab **text → text** hai. Is liye ek hi model sab kar leta hai.

Salahiyat kahan se aati hai? (4 cheezen)

Data (kitna aur kaisa) · **Compute** (training ki taqat) · **Model size** (parameters) · **Training ka tareeqa** (pretraining + post-training)

> **Notice karo:** kabhi kabhi model *"bilal ka ghar lahore mein hai"* jaisa jumla bana deta hai — jo data mein **kahin nahi likha**, magar grammar ke hisaab se bilkul 'munasib' lagta hai. Yahi **hallucination** ki jarr hai: model probable text banata hai, verified text nahi.

🎤 Interview Sawal

Pehle khud jawab socho, phir Jawab par click karo.

Q1. LLM ko ek line mein define karo.

JAWAB

Ek neural network jo bohot bare text par train hua hai aur **pichle tokens dekh kar agla token predict** karta hai; yeh step bar bar dohra kar (generation loop) poora jawab banata hai.

Q2. LLM database ya search engine kyun nahi hai?

JAWAB

Database exact records store karta aur wapas deta hai; search engine documents dhoondta hai. LLM knowledge ko weights mein *compressed patterns* ki shakal mein rakhta hai aur har jawab naye sire se **generate** karta hai — is liye woh ghalat magar confident jawab bhi de sakta hai aur kisi source ki guarantee nahi deta.

Q3. Ek hi model summary, translation, coding aur classification kaise kar leta hai?

JAWAB

Kyunke yeh sab **text-in → text-out** kaam hain. Pretraining mein model ne in sab tarah ke patterns dekhe hain, aur prompt batata hai ke is waqt kaunsa pattern chahiye.

Q4. LLM ki salahiyat kin cheezon se aati hai?

JAWAB

Chaar: **data** (miqdar aur quality), **compute**, **model size (parameters)** aur **training method** (pretraining + SFT/RLHF jaisi post-training). Sirf size barhane se hamesha behtari nahi aati — data aur training method utne hi aham hain.

Q5. Hallucination ki bunyadi wajah kya hai?

JAWAB

Model ka objective 'munasib agla token' hai, 'sach agla token' nahi. Jab use asal jawab nahi pata, tab bhi woh grammar aur style ke hisaab se sab se probable text bana deta hai — jo confident lagta hai magar ghalat ho sakta hai.

1.2 Tokenization

📘 Concept

Model **lafz nahi, tokens** parhta hai. Token = text ka chhota tukra: poora lafz, lafz ka hissa, ya ek nishaan.

```
Text  →  Tokenizer  →  Tokens  →  Token IDs (numbers)  →  Model
"unhappiness"  →  ["un", "happiness"]  →  [403, 67021]
```

Term	Matlab
Token	Text ka tukra. Andaza: English mein **1 token ≈ 4 characters ≈ ¾ lafz**
Token ID	Har token ka number — model asal mein numbers ki list dekhta hai
Tokenizer	Text ko tokens mein todne wala tool. **Har model family ka alag** — ek hi text ke tokens alag ho sakte hain
BPE (Byte Pair Encoding)	Aam tareeqa: aksar saath aane wale characters ko jor kar tokens banana

Yeh kyun zaroori hai? Tokens = Paisa + Jagah + Waqt

- **Cost:** API ka bill tokens par (input aur output alag rate)
- **Context window:** window tokens se bharti hai, lafzon se nahi
- **Latency:** zyada output tokens = zyada waqt

Kin cheezon par zyada tokens lagte hain

English kam leti hai. **Urdu script, Roman Urdu/Hinglish, nayab alfaaz, numbers, code aur emojis** zyada tokens lete hain — kyunke tokenizer zyada tar English data par bana hai.

Spelling & counting failures

Model ko "strawberry" ek-do tokens ki shakal mein dikhta hai, alag alag harf nahi. Is liye *"strawberry mein kitne r hain?"* ya lambe numbers ki math mein ghalti hoti hai.

process behind LLM

```
Aap ka text  ──►  OpenAI/Claude server
                    1. Tokenize (text → token IDs)
                    2. Model chalta hai
                    3. Detokenize (token IDs → text)
user         ◄──  Jawab text mein wapis
```

🎤 Interview Sawal

Pehle khud jawab socho, phir Jawab par click karo.

Q1. Token kya hai aur model lafzon ke bajaye tokens kyun istemal karta hai?

> **JAWAB**
>
> Token text ka chhota tukra hai (lafz, lafz ka hissa ya nishaan). Har lafz ko alag ID dene se vocabulary lamehdood ho jati; characters par chalne se sequence bohot lambi. **Subword tokens** beech ka raasta hain — mehdood vocabulary (~100k–200k) mein har naya lafz bhi tukron se ban jata hai.

Q2. Do texts ka word count barabar hai, phir bhi ek ki API cost double hai. Kyun?

> **JAWAB**
>
> Cost **tokens** par hai, words par nahi. Urdu script, Roman Urdu, code, numbers, emojis ya nayab alfaaz zyada tokens mein tootte hain kyunke tokenizer zyada tar English par bana hai. Tokenizer bhi model ke saath badalta hai.

Q3. Model 'strawberry mein kitne r hain' mein ghalti kyun karta hai?

> **JAWAB**
>
> Model ko alag alag characters nazar nahi aate — use 'straw' + 'berry' jaise 1–3 tokens dikhte hain. Character-level ginti uske input mein seedhi maujood hi nahi. Isi wajah se lambe numbers ki arithmetic bhi kamzor hoti hai.

Q4. Tokens system ki kin teen cheezon ko seedha mutasir karte hain?

> **JAWAB**
>
> **Cost** (bill per token), **context window** (kitna fit hoga) aur **latency** (har output token ek ek kar ke banta hai).

Q5. Multilingual (Urdu/Roman Urdu) app ke liye tokenization ka kya asar hai?

> **JAWAB**
>
> Same baat ke zyada tokens → zyada kharcha, context jaldi bharta hai, aur thora zyada latency. Model chunte waqt apne asal text par token count naapna chahiye; naye tokenizers (jaise bara vocabulary) aksar non-English par behtar compression dete hain.

1.3 Inside an LLM: Embeddings, Attention & Transformers

📘 Concept — Token se agle token tak ka safar

note

hum jaha pa transfomrer ka concept para gaa register sa

🎤 Interview Sawal

Pehle khud jawab socho, phir Jawab par click karo.

Q1. Embedding kya hai aur yeh kyun kaam karti hai?

JAWAB

Token (ya text) ko ek dense vector mein badalna jahan **faasla = matlab ka farq**. Training ke dauran milte julte context mein aane wale tokens ke vectors qareeb aa jate hain, is liye 'car' aur 'gaari' qareeb, 'biryani' door.

Q2. Attention ko Q, K, V ke saath samjhao.

JAWAB

Har token se teen vectors bante hain. Token ki **Query** baqi sab tokens ki **Keys** se dot-product kar ke score banati hai ('kaun mere kaam ka hai?'). Scores ko $\sqrt{d_k}$ se scale kar ke softmax se weights banate hain, phir un weights se **Values** ka weighted average — yeh token ka naya, context-aware representation hai. Multi-head attention yahi kaam kayi 'heads' mein parallel karta hai taake mukhtalif rishte (grammar, reference, waghera) pakre ja sakein.

Q3. $\sqrt{d_k}$ se divide kyun karte hain?

JAWAB

Bare d_k mein dot-products ki variance barh jati hai, scores bohot bare ho kar softmax ko almost one-hot bana dete hain → gradients bohot chhote. Scaling se distribution stable rehti hai aur training achhi hoti hai.

Q4. Causal attention kya hai aur decoder LLMs mein kyun zaroori hai?

JAWAB

Mask lagta hai taake token sirf apne se **pehle** wale tokens ko dekhe. Training mein model agla token predict karta hai — agar aage dekh sakta to 'cheating' hoti. Inference par aage ka text maujood hi nahi hota.

Q5. Dense vs Mixture-of-Experts (MoE) — faida aur nuqsan?

JAWAB

Dense: har token par saare parameters → simple, magar compute mehnga. MoE: router har token ko chand experts par bhejta hai → **total capacity zyada, per-token compute kam** (tez/sasta). Nuqsan: saare experts phir bhi **memory mein** rakhne parte hain, routing/load-balancing pechida, aur serving infra mushkil.

Q6. Kya bara model hamesha behtar hai?

JAWAB

Nahi. Bara model aksar zyada qabil, magar **mehnga, slow aur zyada GPU memory** wala. Tang kaam (classification, extraction) par chhota ya specialized model barabar ya behtar ho sakta hai. Faisla apne kaam par naap kar (1.12).

Q7. 70B model FP16 mein chalane ke liye kitni GPU memory chahiye?

JAWAB

Sirf weights: 70 × 2 bytes ≈ **140 GB**, upar se KV-cache aur activations. Is liye multi-GPU ya quantization (INT4 ≈ 35 GB) chahiye.

1.4 How LLMs Learn

📘 Concept — do bare marahil

```
[1] PRETRAINING                          [2] POST-TRAINING
Internet ka bohot bara text      →       SFT  →  Preference / RL (RLHF, RLAIF, DPO)  →  Deploy
"agla token predict karo"                "madadgar, mehfooz assistant bano"
Natija: BASE model                       Natija: INSTRUCT / CHAT model
```

Term	Matlab
Pretraining	Trillions tokens par next-token prediction. Grammar, facts, coding syntax, reasoning patterns yahin se
Self-supervised learning	Label insaan nahi lagata — **text khud hi jawab hai** (agla lafz). Is liye itne bare data par training mumkin
Memorization vs Generalization	Ratta vs asal pattern. Acha model naye inputs par bhi chalta hai
SFT (Supervised Fine-Tuning)	Achhe (instruction → jawab) examples par training taake model hidayat follow kare
RLHF / RLAIF	Insaan (ya AI) do jawabon mein behtar chunta hai → reward model → RL se model us taraf
DPO	Preference pairs se seedha training, alag reward model aur RL loop ke baghair — aasan aur stable
Safety tuning	Nuqsandeh requests par inkaar sikhana

Base vs Instruct — farq ek misaal se

Prompt: **"Pakistan ka capital kya hai?"**

Base model	Instruct model
"Pakistan ka capital kya hai? India ka capital kya hai? Bangladesh ka...?" — sirf text aage barhata hai (shayad quiz page jaisa pattern dekha ho)	*"Pakistan ka capital Islamabad hai."* — sawal ko sawal samajh kar jawab

Base model **bohot kuch jaanta hai**, magar assistant ki tarah *behave* karna post-training sikhati hai.

🎤 Interview Sawal

Pehle khud jawab socho, phir Jawab par click karo.

Q1. Pretraining aur fine-tuning mein farq?

JAWAB

Pretraining: bohot bare, ghair-labeled data par next-token prediction — general knowledge aur language patterns. Mehnga (hazaron GPUs, hafton/mahinon). **Fine-tuning (SFT waghera):** chhote, curated data par mazeed training taake model khaas behavior seekhe (instructions follow karna, format, tone). Fine-tuning naye facts dalne ka acha tareeqa nahi — uske liye RAG.

Q2. Pretrained (base) model itna kuch jaanta hai, phir bhi kamzor assistant kyun hai?

JAWAB

Uska objective sirf 'text aage barhao' tha. Use sawal ka jawab dena, ruk jana, inkaar karna ya clarification maangna nahi sikhaya gaya. Yeh behaviour **post-training** (SFT + preference tuning) se aata hai.

Q3. 'Self-supervised' ka kya matlab hai?

JAWAB

Labels data se khud bante hain: text ka har agla token hi target hai. Insaani labeling ki zaroorat nahi, is liye internet-scale data istemal ho sakta hai.

Q4. RLHF ke steps batao.

JAWAB

1) SFT model bana lo. 2) Ek prompt par kayi jawab generate karo; insaan unhein rank kare. 3) In rankings se **reward model** train karo. 4) RL (aam tor par PPO) se LLM ko optimize karo ke reward zyada mile, magar original model se zyada door na jaye (KL penalty).

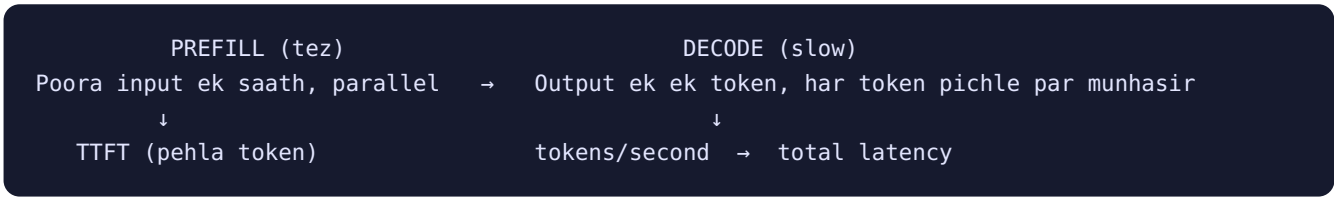
Q5. DPO, RLHF se kaise mukhtalif hai?

JAWAB

DPO alag reward model aur RL loop nahi chalata. Seedha (prompt, pasandeeda, radd shuda) pairs par ek loss se model ko pasandeeda jawab ki probability barhane par train karta hai. Aasan, sasta aur zyada stable — is liye bohot popular.

1.5 How LLM Inference Works

📘 Concept — prompt bhejne ke baad kya hota hai



Term	Matlab
Inference	Trained model se jawab nikalwana
Prefill	Saare input tokens ek saath process (GPU parallel) — tez
Decode	Output tokens **sequentially** — har naye token ke liye ek forward pass. Yeh asal slow hissa
TTFT (Time to First Token)	Pehla token aane mein waqt — user ko 'jawab shuru' hone ka ehsaas
Tokens/second	Decode ki raftar
KV caching	Pichle tokens ke Key/Value vectors save — har naye token par poori history dobara calculate nahi. Bacha compute, magar **memory** khaata hai (lambe context = bara KV cache)

Total latency ≈ TTFT + (output tokens ÷ tokens per second)

Cost drivers

Lambe prompts (input tokens) · lambe outputs (output tokens — aam tor par **mehnge**) · reasoning models (chhupe 'thinking' tokens, 1.7)

🎤 Interview Sawal

Pehle khud jawab socho, phir Jawab par click karo.

Q1. Prefill aur decode mein farq? Kaunsa slow hai aur kyun?

> **JAWAB**
>
> **Prefill** saare input tokens ek hi forward pass mein parallel process karta hai (compute-bound, tez). **Decode** har output token ke liye alag forward pass karta hai kyunke har token pichle par munhasir hai — sequential hai, is liye slow (aam tor par memory-bandwidth bound).

Q2. 2,000 output tokens banana 50 se itna zyada waqt kyun leta hai, jabke input wahi ho?

> **JAWAB**
>
> Kyunke decode sequential hai: latency ≈ TTFT + output_tokens / tokens_per_sec. 40 guna zyada tokens = taqreeban 40 guna zyada decode time.

Q3. KV cache kya hai aur iska trade-off kya hai?

> **JAWAB**
>
> Pehle se process hue tokens ke attention Keys/Values ko save rakhna taake har naye token par unhein dobara compute na karna pare. **Faida:** bohot kam compute, tez decode. **Qeemat:** GPU memory — context length aur concurrent users ke saath linearly barhti hai; long-context serving ki asal rukawat yahi hai.

Q4. TTFT kyun aham hai aur isay kaise behtar karte hain?

> **JAWAB**
>
> User ko app ki 'tezi' TTFT se mehsoos hoti hai. Behtar: **streaming**, chhota prompt (kam prefill), prompt caching, chhota/tez model, aur nazdeek region.

Q5. Output tokens aam tor par input se mehnge kyun hote hain?

> **JAWAB**
>
> Input prefill mein parallel aur sasta process hota hai; har output token ek alag sequential forward pass maangta hai, GPU ka zyada waqt leta hai.

1.6 Sampling & Generation Parameters

📘 Concept

Model har step par poore vocabulary ke liye **logits** (raw scores) deta hai → softmax → probabilities. Ab **asal token kaise chuna jaye** — yeh sampling tay karti hai.

Parameter	Kya karta hai	Kab
Greedy	Hamesha sab se zyada probability wala	Predictable, magar dohrao aur boring text
Temperature	Logits ko T se divide → T kam = 'peaky', T zyada = 'flat'	**0–0.3:** extraction, classification, facts · **0.7–1.0:** creative, brainstorming
Top-k	Sirf top k tokens mein se chuno	Lambi 'ajeeb' tail kaatna
Top-p (nucleus)	Sab se chhota set jis ki mila kar probability ≥ p	Dynamic — model yaqeeni ho to kam options, ghair-yaqeeni to zyada
Max output tokens	Jawab ki lambai ki had	Cost/latency control (stop reason 'length' check karo)
Stop sequences	In alfaaz par ruk jao	Format control
Frequency / presence penalty	Dohrane par saza (frequency = jitni bar aaya, presence = aaya ya nahi)	Repetition kam karna

Practical rule: temperature *ya* top-p mein se aam tor par ek hi tune karo, dono ek saath nahi. **Reasoning models** aksar temperature jaise knobs par kam control dete hain.

🎤 Interview Sawal

Pehle khud jawab socho, phir Jawab par click karo.

Q1. Temperature mathematically kya karta hai?

JAWAB

Softmax se pehle logits ko T se divide karta hai: $p_i = e^{z_i/T} / \Sigma_j e^{z_j/T}$. **T < 1:** farq barhta hai, distribution peaky (predictable). **T > 1:** distribution flat (zyada random). **T → 0:** greedy jaisa.

Q2. Kya temperature 0 par output hamesha bilkul ek jaisa aata hai?

JAWAB

Aam tor par *qareeb qareeb*, magar guarantee nahi. GPU floating-point, batching aur provider-side changes se chhote farq aa sakte hain. Is liye evals mein ek run par bharosa nahi karte.

Q3. Top-k aur top-p mein farq?

JAWAB

Top-k hamesha fixed k tokens rakhta hai. **Top-p** utne tokens rakhta hai jitne mil kar p probability banayein — jab model yaqeeni ho to 1–2, ghair-yaqeeni ho to zyada. Top-p zyada adaptive hai.

Q4. JSON extraction ke liye aur marketing taglines ke liye kya settings rakhoge?

JAWAB

Extraction: **temperature 0–0.2**, structured output/schema, mukarrar max_tokens. Taglines: **temperature ~0.8–1.0** (ya top-p ~0.9), kayi variations generate kar ke behtareen chuno.

Q5. Frequency aur presence penalty mein farq?

JAWAB

Frequency: jitni zyada bar token aa chuka, utni zyada saza → lafzi dohrao kam. **Presence:** token ek bar bhi aa gaya to fixed saza → naye mauzoo ki taraf dhakelna.

1.7 Reasoning Models & Test-Time Compute

📘 Concept

Kuch models jawab dene se pehle **'sochte' hain** — andar hi andar reasoning tokens banate hain. Is ko **test-time compute** kehte hain: training ke bajaye *jawab dete waqt* zyada computation laga kar behtar nateeja.

Term	Matlab
Test-time compute	Inference par zyada tokens/steps kharch kar ke quality barhana
Reasoning effort / thinking budget	Kitna sochne dena hai — zyada = behtar magar **slow aur mehnga**
Internal reasoning tokens	Aksar aap ko nazar nahi aate, **magar bill mein shamil** hote hain aur latency barhate hain

✅ Reasoning faida deti hai	❌ Faltu kharcha
Mushkil math, complex coding, planning, multi-step logic, bohot si shartain	Sentiment, simple extraction, rewriting, seedhi summary

Reasoning models ko prompt kaise karein: "step by step socho" kehne ki zaroorat nahi — **goal, context, shartain aur acha jawab kaisa dikhta hai** saaf batao.

🎤 Interview Sawal

Pehle khud jawab socho, phir Jawab par click karo.

Q1. Test-time compute kya hai?

JAWAB

Jawab dete waqt zyada computation istemal karna — jaise model ka pehle andar reasoning tokens banana, kayi jawab bana kar behtareen chunna, ya apna kaam verify karna. Training ke baad bhi quality barhane ka ek 'knob'.

Q2. Reasoning model kab chunoge aur kab nahi?

JAWAB

Haan: multi-step math/logic, mushkil coding/debugging, planning, bohot si shartain wale faisle.
Nahi: classification, extraction, rewriting, simple Q&A — wahan latency aur kharcha barhta hai, quality nahi. Faisla apne eval set par naap kar.

Q3. Reasoning tokens nazar nahi aate to unka bill kyun?

> **JAWAB**
>
> Kyunke model ne unhein generate kiya — GPU time laga. Provider aam tor par unhein output tokens ki tarah charge karta hai. Is liye cost estimate mein 'visible output' ke ilawa reasoning budget bhi gino.

Q4. Reasoning model ko prompt karna normal model se kaise mukhtalif hai?

> **JAWAB**
>
> Chain-of-thought trick ('step by step socho') ki zaroorat nahi — model khud sochta hai. Behtar hai **saaf goal, context, constraints aur success criteria** dein, aur bohot zyada micro-instructions se bachein.

1.8 The Context Window

📘 Concept

Context window = tokens ki zyada se zyada tadaad jo model **ek request** mein dekh sakta hai — **input + output dono mila kar**. Yeh model ki 'working memory' hai.

Window kya kya bharta hai

System instructions · conversation history · user prompt · documents (RAG) · few-shot examples · tool definitions & outputs · **khud model ka generated jawab**

Teen alag cheezen — interview mein confuse mat karna

	Kahan se	Misaal
Knowledge	Training mein seekha (weights)	"Python list kya hai"
Context	Is request mein bheja gaya	Aap ka diya hua document
Memory	App ne save kar ke dobara context mein dala	"User ka naam Ali hai" — DB se

API stateless hai: model pichli request yaad nahi rakhta — har bar poori history *aap* bhejte hain.

Masla	Matlab
Truncation	Window bhar jaye to purana text kaatna parta hai (ya request fail)
Lost in the middle	Lambe context ke **darmiyan** wali baat model aksar nazar andaaz karta hai; shuru aur aakhir behtar yaad
Context dilution	Faltu text se zaroori baat 'pani' ho jati hai, dhyan bikhar jata hai

Bari window ≠ masla hal. Aksar behtar strategy yeh hai ke **sahi aur chhota** context chuno (RAG, summary) — bara model nahi.

🎤 Interview Sawal

Pehle khud jawab socho, phir Jawab par click karo.

Q1. Context window kya hai aur us mein kya kya count hota hai?

JAWAB

Ek request mein model jitne tokens dekh sakta hai — **input aur output mila kar**. Is mein system prompt, history, user message, documents, examples, tool definitions/results aur model ka apna output sab shamil.

Q2. Context, knowledge aur memory mein farq?

JAWAB

Knowledge training se weights mein hai (cutoff tak). **Context** is request mein bheja gaya text. **Memory** app-level feature: app maloomat save kar ke baad ki requests ke context mein wapas dalti hai. Model khud request ke darmiyan kuch yaad nahi rakhta (stateless).

Q3. 1M token context window hai — kya RAG ki zaroorat khatam?

JAWAB

Nahi. (1) **Cost aur latency** — har request par lakhon tokens bhejna mehnga aur slow. (2) **Lost-in-the-middle / dilution** — bari window mein har token par barabar dhyan nahi. (3) Data window se bhi bara ho sakta hai aur badalta rehta hai. (4) Access control aur citations. Bari window kuch cases aasan karti hai, magar selective retrieval aksar sasta aur zyada accurate hai.

Q4. 'Lost in the middle' kya hai aur kaise kam karoge?

JAWAB

Lambe context ke beech ki maloomat model kam istemal karta hai. Ilaj: aham maloomat **shuru ya aakhir** mein rakho, context chhota karo (sirf top relevant chunks), reranking, aur saaf sections/tags.

Q5. Chatbot ki conversation lambi ho kar window se bari ho jaye to kya karoge?

JAWAB

Options: purane messages hatana (sliding window), purani history ki **summary**, aham facts ko memory mein save kar ke sirf zaroorat par retrieve karna. System prompt aur current task hamesha mehfooz rakhna.

1.9 LLM Capabilities & Failure Modes

📘 Concept

✅ Taqat

Summary · text ko badalna (rewrite/tone) · extraction · classification · matlab samajhna · coding · translation · brainstorming · **few-shot se seekhna** (chand examples dekh kar pattern pakarna)

❌ Kahan dhoka deta hai

Failure	Wajah	Ilaj (aage ke levels)
Hallucination	'Munasib' text, verified nahi	RAG + grounding, citations, abstention ("nahi pata")
Arithmetic & brittle reasoning	Tokens, digits nahi; patterns, algorithm nahi	**Tools** (calculator/code), reasoning model
Prompt sensitivity	Chhota lafzi farq → alag jawab	Prompt testing on a test set (L3)
Nondeterminism	Sampling + infra	Low temperature, evals ko kayi bar chalana
Outdated knowledge	Training cutoff	Retrieval / web search tools

> **Sab se aham soch: "Kya kar sakta hai?" nahi — "Meri app ke halaat mein *kitne bharose* se karta hai?"**

Demo mein 10 mein 10 sahi ≠ production mein 1,000 mein 1,000. Reliability ko **naapo** (L7 Evals).

🎤 Interview Sawal

Pehle khud jawab socho, phir Jawab par click karo.

Q1. LLM hallucinate kyun karta hai aur production mein isay kaise kam karoge?

JAWAB

Objective probable text hai, sach nahi; aur model ko 'nahi pata' kehne ke liye khaas tor par train na kiya ho to woh khala bhar deta hai. **Kam karne ke tareeqe:** RAG se grounding, 'sirf diye gaye context se jawab do' hidayat, citations, abstention ki ijazat, low temperature, structured output + validation, aur hassas cheezon par human review. Isay **evals** se naapo.

Q2. LLM math mein kamzor kyun hai? Production fix?

> **JAWAB**
>
> Numbers tokens mein tootte hain aur model algorithm nahi, pattern follow karta hai. Fix: **tool calling** (calculator/Python) ya code execution — model sirf tay kare kya calculate karna hai.

Q3. Kisi kaam ke liye LLM ki feasibility kaise judge karoge?

> **JAWAB**
>
> Sawal 'kya kar sakta hai' ke bajaye '**kitni reliability se**': apne asli inputs ka test set banao (normal + edge cases), accuracy/consistency/latency/cost naapo, failure types dekho, aur tay karo kin cases par safeguards (validation, human review) chahiye.

Q4. Nondeterminism ka evaluation par kya asar hai?

> **JAWAB**
>
> Ek run ke nateeje par faisla ghalat ho sakta hai. Kayi runs chalao, averages/variance dekho, tolerance bands rakho, aur jahan mumkin ho temperature kam karo.

1.10 Understanding the Model Landscape

📘 Concept

Qism	Misaal	✅ Faida	❌ Nuqsan
Closed / API models	GPT, Claude, Gemini families	Frontier quality, zero infra, jaldi shuru	Vendor par inhisar, data bahar jata hai, per-token kharcha
Open-weight models	Llama, Qwen, Mistral, Gemma families	Poora control, privacy, fine-tuning, scale par sasta	GPU/infra ki mehnat, serving expertise
Managed platforms	AWS Bedrock, Google Vertex AI, Azure AI Foundry	Apne cloud account ke andar — compliance, IAM, ek bill	Catalog thora peeche, platform lock-in
Inference providers	Open models host kar ke API dene wali companies	Open model, bina GPU sambhale	Teesri party par data, provider ke limits

Aur kya jaanna zaroori hai

- **Small vs large, general vs specialized:** har kaam ke liye sab se bara model nahi — chhota specialized aksar behtar *aur* sasta.
- **Knowledge cutoff:** model ki maloomat ki aakhri tareekh. Nayi maloomat → context, retrieval ya tools.
- **Versions & deprecation:** models ke **dated snapshots** aate hain aur purane band hote hain → production mein **version pin karo** aur naye version par **dobara eval** chalao.
- **Open-weight licensing:** permissive, restricted-use ya non-commercial — "open" ka matlab hamesha "business ke liye free" nahi. Pehle license parho.

🎤 Interview Sawal

Pehle khud jawab socho, phir Jawab par click karo.

Q1. Closed API model aur open-weight model mein kab kaunsa chunoge?

JAWAB

Closed API: jaldi launch, frontier quality chahiye, team chhoti, data sharing qabil-e-qubool. **Open-weight:** data bahar nahi ja sakta, fine-tuning/poora control chahiye, bohot zyada volume (jahan apna GPU sasta pare), ya offline/edge. Aksar hybrid: mushkil kaam closed, aasan/high-volume kaam open.

Q2. Production mein model version pin karna kyun zaroori hai?

> **JAWAB**
>
> Provider 'latest' alias ke peeche model badal sakta hai → baghair code badle behavior badal jata hai (silent regression). Pinned snapshot se system reproducible rehta hai; naye version par jaane se pehle eval suite chalao.

Q3. Knowledge cutoff kya hai aur uske baad ki maloomat model ko kaise doge?

> **JAWAB**
>
> Training data ki aakhri tareekh — us ke baad ki cheezen model nahi jaanta. Nayi maloomat **context** mein (RAG, web search tool, ya seedha prompt mein) di jati hai, fine-tuning se nahi.

Q4. 'Open-weight' aur 'open-source' ek hi cheez hain?

> **JAWAB**
>
> Nahi. Open-weight ka matlab weights download ho sakte hain; training data/code aksar open nahi hota, aur license commercial use par shartain laga sakta hai. Business mein istemal se pehle license check karna zaroori.

1.11 Benchmarks & Leaderboards

📘 Concept

Benchmark = sawalon ka fixed set + fixed scoring — models ka imtihan.

Naam	Kya naapta hai
MMLU	~57 subjects ke MCQs — general knowledge
GPQA	PhD-level mushkil science sawal ("Google-proof")
LMArena (Chatbot Arena)	Log do anjaan models ke jawab dekh kar behtar chunte hain → **human preference** ranking (Elo jaisa)

Number ko tanqeedi nazar se kyun parhein

Masla	Matlab
Rank doesn't transfer	Leaderboard ka prompt, task aur 'acha jawab' aap ke kaam se alag
Data contamination	Benchmark ke sawal training data mein aa gaye — 'paper leak'
Saturation	Sab 90%+ le rahe hon to benchmark farq nahi bata sakta
Overfitting	Public test ke liye tune kar ke score barhana, asal salahiyat nahi

Golden rule: **Benchmarks se shortlist, apne data par faisla.**

🎤 Interview Sawal

Pehle khud jawab socho, phir Jawab par click karo.

Q1. Model A leaderboard par #1 hai. Kya tum use apni app ke liye chun loge?

> **JAWAB**
>
> Seedha nahi. Leaderboard ke tasks, prompts aur scoring meri app se alag hain, aur contamination/ overfitting mumkin hai. Leaderboard se **2–3 models shortlist** karunga, phir **apne asli examples** par quality, latency aur cost naap kar faisla.

Q2. Data contamination kya hai?

> **JAWAB**
>
> Benchmark ke sawal/jawab model ke training data mein shamil ho jana — model ne 'ratta' laga liya, is liye score asal salahiyat se zyada dikhta hai.

Q3. MMLU aur LMArena mein farq?

> **JAWAB**
>
> **MMLU** fixed MCQ imtihan — automatic scoring, knowledge naapta hai. **LMArena** insaan do anjaan models ke jawab compare karte hain — human preference naapta hai (tone, madadgari bhi shamil). Dono alag cheezen naapte hain; dono aap ke task ka seedha andaza nahi.

Q4. Benchmark 'saturated' hone ka kya matlab hai?

> **JAWAB**
>
> Zyada tar top models taqreeban full marks le rahe hain, to benchmark ab un mein farq nahi kar sakta. Is liye naye, mushkil benchmarks (jaise GPQA, HLE) aate rehte hain.

1.12 Choosing the Right Model

📘 Concept — 8 dimensions

#	Dimension	Sawal
1	**Capability**	Kya yeh is qism ka kaam kar sakta hai?
2	**Quality (aap ke kaam par)**	Mere eval set par score?
3	**Latency**	TTFT aur total time — p95 par
4	**Cost**	Per request (input + output + reasoning tokens)
5	**Context size**	Mera max input fit hota hai?
6	**Modalities**	Text / image / audio / video chahiye?
7	**Privacy & deployment**	Data kahan ja sakta hai? API, cloud platform ya self-host?
8	**Product features**	Structured output, tool calling, streaming, caching, batch

Model card & pricing page = official sach: salahiyat, limits, context window, qeemat per million tokens.

Selection workflow

```
Kaam define → Requirements (hard limits) → Shortlist (benchmarks/cards) → Apne examples par test
→ Quality + latency + cost naapo → Failures parho → Model chuno (aur version pin karo)
```

🎤 Interview Sawal

Pehle khud jawab socho, phir Jawab par click karo.

Q1. Customer-support bot ke liye model kaise chunoge? (classic interview sawal)

JAWAB

1) **Requirements:** zubanein (Urdu/Roman Urdu?), latency (p95 < ~2s), volume aur budget, privacy (customer data), features (tool calling for order lookup, structured output). 2) **Hard filters** se kuch models bahar. 3) Model cards/benchmarks se **2–3 shortlist**. 4) **Asli support messages** ka eval set (50–200, edge cases samet) — accuracy, groundedness, tone. 5) Latency aur **cost per 1,000 requests** naapo. 6) Failures parho. 7) Aksar chhota, tez model jeetta hai; mushkil cases ko bare model par route kiya ja sakta hai. 8) Version pin, monitoring.

Q2. Model choose karte waqt kin dimensions par sochte ho?

JAWAB

Capability, apne task par quality, latency, cost, context size, modalities, privacy/deployment, aur product features (structured output, tools, streaming, caching).

Q3. Kya hamesha sab se qabil (frontier) model lena chahiye?

JAWAB

Nahi — woh aksar sab se mehnga aur slow hota hai. Sab se sasta/tez model jo **meri quality threshold** paar kare, wohi sahi hai. Zaroorat ho to routing: aasan requests chhote model, mushkil bare model par.

Q4. Model ki asli cost kaise nikaloge?

JAWAB

Per request: (avg input tokens × input rate) + (avg output tokens + reasoning tokens) × output rate, phir × monthly requests. Caching discounts, retries aur system prompt ka size shamil karo. Yeh number pricing page aur apne token counts se aata hai, andaze se nahi.

📝 Revision Sheet — ek nazar mein (interview se pehle 5 minute)

Module	Ek line jo yaad rakhni hai
1.1	LLM = **next-token predictor** in a loop (predict → choose → append → repeat). 'Munasib', na ke 'sach'.
1.2	Model **tokens** dekhta hai. Tokens = cost + context + latency. Urdu/Roman Urdu/code/numbers zyada tokens.
1.3	Embedding = matlab ka vector. Attention = softmax($QK^T/\sqrt{d_k}$)V. Causal mask. MoE = zyada total, kam active params (memory phir bhi poori).
1.4	Pretraining (self-supervised) → base model. SFT + RLHF/DPO → instruct model.
1.5	Prefill tez/parallel, decode slow/sequential. Latency ≈ TTFT + out/TPS. KV cache compute bachata, memory khata.
1.6	Temperature = logits ÷ T. Top-k fixed, top-p adaptive. Extraction T≈0, creative T≈0.8+.
1.7	Reasoning = test-time compute. Chhupe tokens bhi bill. Sirf mushkil kaam par.
1.8	Window = input + output. API stateless. Lost-in-the-middle. Bari window ≠ RAG khatam.
1.9	Hallucination, math, prompt sensitivity, nondeterminism, cutoff. Sawal: **"kitne bharose se?"**
1.10	Closed API vs open-weight vs managed platform vs inference provider. Version pin. License check.
1.11	Benchmarks se **shortlist**, apne data par **faisla**. Contamination, saturation.
1.12	8 dimensions. Hard constraints → shortlist → own eval → quality/latency/cost → choose.

⚠️ Aam ghaltiyan jo interview mein nahi karni

- "LLM sochta/jaanta hai" — kaho: **patterns se probable tokens predict karta hai**
- "Temperature 0 = 100% deterministic" — *aam tor par qareeb*, guarantee nahi
- "Bara context hai to RAG ki zaroorat nahi" — cost, latency, dilution, freshness
- "Leaderboard #1 = best for us" — apne eval ke baghair nahi
- "Fine-tune kar ke naye facts sikha denge" — facts ke liye **RAG**; fine-tuning behavior/format ke liye
- "MoE kam memory leta hai" — compute kam, **memory nahi**

⚡ Rapid-Fire (24 sawal) — 1 line mein jawab do

1. LLM ka training objective?

> Next-token prediction (cross-entropy loss on the next token).

2. English mein 1 token ≈ ?

> ~4 characters ≈ ¾ lafz.

3. Token IDs kya hain?

> Har token ka vocabulary mein number; model inhi numbers ko embeddings mein badalta hai.

4. Attention formula?

> $\text{softmax}(QK^T / \sqrt{d_k}) \cdot V$

5. Causal mask kyun?

> Taake token aage ke tokens na dekhe — training mein cheating na ho, inference ke mutabiq rahe.

6. Dense vs MoE ek line?

> Dense: har token par saare params. MoE: router chand experts chunta hai — compute kam, memory nahi.

7. 70B FP16 weights memory?

> ~140 GB.

8. Base vs instruct?

> Base sirf text continue karta hai; instruct SFT/RLHF ke baad hidayat follow karta hai.

9. RLHF ke 3 bare steps?

> SFT → reward model (human rankings) → RL (PPO) with KL penalty.

10. DPO ka faida?

> Reward model aur RL loop ke baghair preference pairs se seedhi, stable training.

11. Prefill vs decode?

> Prefill: input parallel, tez. Decode: output sequential, slow.

12. TTFT?

> Time to first token — perceived speed; streaming se behtar mehsoos hota hai.

13. KV cache trade-off?

> Compute bachata hai, GPU memory khata hai (context × users ke saath barhta hai).

14. Temperature barhane se kya?

> Distribution flat → zyada random/creative, zyada ghaltiyan.

15. Top-p = 0.9 ka matlab?

> Sab se chhota token set jis ki cumulative probability ≥ 0.9, usi se sample.

16. Reasoning tokens ka bill?

> Haan — aam tor par output rate par, chahe nazar na aayein.

17. Context window mein output count hota hai?

> Haan — input + output dono.

18. API stateless ka matlab?

> Model request ke darmiyan kuch yaad nahi rakhta; history app bhejti hai.

19. Lost in the middle?

> Lambe context ke darmiyan ki maloomat model kam istemal karta hai.

20. Math ka production fix?

> Tool calling — calculator/code execution.

21. Knowledge cutoff ke baad ki info kaise?

> Context: RAG, web search tools, ya prompt mein.

22. Version pin kyun?

> Silent model updates se regression rokna, reproducibility.

23. Data contamination?

> Benchmark sawal training data mein — score fake taur par zyada.

24. Model selection ka golden rule?

> Benchmarks se shortlist, apne data par measured faisla.

✅ Level 1 Outcome checklist

Neeche ki har cheez kisi dost ko **apne alfaaz mein** samjha sako to Level 1 mukammal:

- ☐ LLM pichle tokens dekh kar bar bar agla munasib token predict kar ke text banata hai
- ☐ Barabar word count wale do texts ke tokens aur qeemat bohot mukhtalif kyun ho sakte hain
- ☐ Text → Tokens → Embeddings → Transformer layers + Attention → Next-token probabilities
- ☐ Pretrained model bohot kuch jaanta hai magar phir bhi kamzor assistant kyun hai
- ☐ 2,000 output tokens 50 se bohot zyada waqt kyun lete hain
- ☐ Kaam ke hisaab se generation settings chunna
- ☐ Kab extra reasoning compute ki qeemat dena faida mand hai
- ☐ Bara context wala model chahiye ya behtar context selection
- ☐ Kaunse kaam LLM ke liye qudrati hain aur kin par safeguards chahiye
- ☐ Do models ko architecture, capability aur deployment ke farq se compare karna
- ☐ Benchmarks ko shortlist ke liye istemal karna, faisle ke liye nahi
- ☐ Model ko application ki zaroorat aur naape gaye nateejon par chunna

Agla: Level 2 — Working with LLM APIs (Orchestration)